Final Project Report

By Shashank Kumar (24BAI10032)

Project Title: RoomSync – Hotel Reservation & Management System

1. Objectives of the Project

- Develop a backend-based Hotel Reservation & Management System using Spring Boot and MongoDB.
- Manage hotel rooms through CRUD operations(Add new rooms to the hotel inventory, view available and occupied rooms, update room details such as room type, price, and status, remove room records when necessary, maintain customer records efficiently.
- Create, update, view, and cancel room bookings.
- Demonstrate entity relationships between rooms, customers, and bookings.
- Develop RESTful APIs using Spring Boot.
- Implement business logic for hotel operations

2. Tech Stack

- Java 17 (Eclipse Temurin LTS)
- Spring Boot 3.x
- Spring Data MongoDB
- MongoDB
- Maven
- Postman
- Git & GitHub

4. REST APIs Used

GET, POST, PUT and DELETE endpoints are implemented to perform CRUD operations.

4. Spring Boot Annotations Used

- @SpringBootApplication
- @RestController
- @RequestMapping
- @GetMapping
- @PostMapping
- @PutMapping
- @DeleteMapping
- @Autowired
- @Service
- @Repository

3. Project Structure

```
src
└──── main
    └──── java
        └──── com
            └──── roomsync
                    │
                    ├── controller
                    │       └──── RoomController
                    │       └──── CustomerController
                    │       └──── BookingController
                    │
                    ├── service
                    │       └──── RoomService
                    │       └──── CustomerService
                    │       └──── BookingService
                    │
                    ├── repository
                    │       └──── RoomRepository
                    │       └──── CustomerRepository
                    │       └──── BookingRepository
                    │
                    ├── model
                    │       └──── Room
                    │       └──── Customer
                    │       └──── Booking
                    │
                    ├── exception
                    │           ├──── ResourceNotFoundException.java
                    │           ├──── RoomNotAvailableException.java
                    │           ├──── BookingOperationException.java
                    │           ├──── ErrorResponse.java
                    │           └──── GlobalExceptionHandler.java
                    │
                    └── RoomSyncApplication
```

5. Entity Relationships

Customer → Booking represents a One-to-Many relationship. Customers and Rooms are conceptually connected through the Booking collection.

One-to-Many Output:


```
[
{
"id": "B001",
"customerId": "C001",
"roomId": "R101",
"checkInDate": "2026-06-20",
"checkOutDate": "2026-06-22",
"bookingStatus": "CHECKED_OUT"
},
{
"id": "B002",
"customerId": "C001",
"roomId": "R102",
"checkInDate": "2026-06-25",
"checkOutDate": "2026-06-28",
"bookingStatus": "CONFIRMED"
}
]
```


6. Business Flow

Customer
↓
Creates Booking
↓
Booking references Room
↓
Room becomes BOOKED
↓
Customer Check-In
↓
Room becomes OCCUPIED
↓
Customer Check-Out
↓
Room becomes AVAILABLE

7. Key Features

- Room Management
- Customer Management
- Booking Management
- Check-In and Check-Out
- Booking Cancellation
- Room Status Management
- Date Validation
- Exception Handling

8. Challenges Faced and Bugs Fixed (Extra done)

Bug 1: booking.getBookingStatus() was treated as a boolean. Since it returns a String, comparison with BookingStatus constants was used.

Bug 2: During check-in, room status was updated to OCCUPIED but booking status was not changed. CHECKED_IN status was added.

Bug 3: During check-out, room status became AVAILABLE but booking status remained unchanged. CHECKED_OUT status was added.

Bug 4: Invalid dates such as Check-In 25 June and Check-Out 20 June were accepted. Validation was introduced to ensure check-out date is after check-in date.

Bug 5: Multiple bookings on unavailable rooms were possible. Room availability validation was implemented to prevent duplicate active bookings.

//All mentioned bugs were fixed while testing on Postman and same can be verified by downloading the Repository Final Project Folder's file / extracted using **Final Project Zipped.zip**

9.Exception Handling and Error Management (Extra work done)

a) 404 – Resource Not Found

This error occurs when a requested resource such as a Room, Customer, or Booking does not exist in the database. Initially, the application was throwing generic runtime exceptions which produced unclear error messages. To improve readability, a custom ResourceNotFoundException class was created. The exception is handled globally using @ControllerAdvice. Whenever a user requests an invalid ID, the system returns a meaningful response with status code 404 and an appropriate message instead of exposing internal errors.

b) 409 – Room Unavailable

This error occurs when a user tries to create a booking for a room that is already booked or occupied. During testing, it was observed that multiple active bookings could be created for the same room. To prevent this issue, room status validation was added before creating a booking. A custom RoomNotAvailableException was introduced to handle this scenario. The system now returns HTTP status code 409 (Conflict) along with a clear message indicating that the room is unavailable.

c) 400 – Invalid Booking Operation

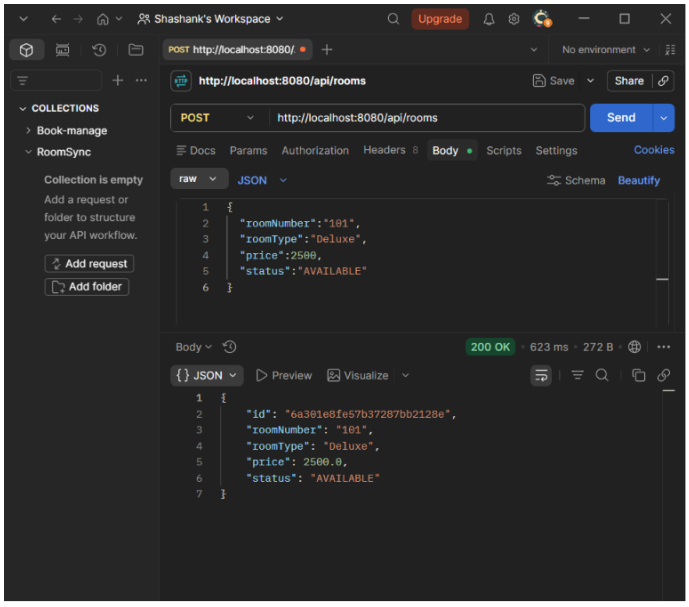
This error is generated when an invalid operation is performed, such as checking out before check-in, cancelling an already cancelled booking, or entering incorrect booking dates. Initially, these invalid states were not restricted. To solve this problem, validations were introduced inside the service layer. A custom BookingOperationException was created to represent business rule violations. Invalid requests are now handled gracefully and return status code 400 with a meaningful explanation.

d) 500 – Internal Server Error

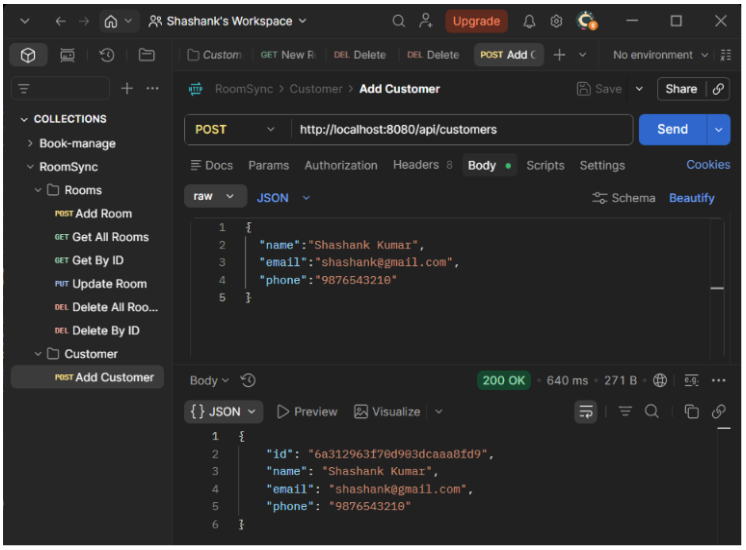
This error represents unexpected failures occurring during application execution. While testing, unhandled exceptions produced long stack traces and unclear responses. To provide a better user experience, a global exception handler was implemented using @ControllerAdvice. Any unexpected exception is captured and converted into a standard JSON response. The application returns status code 500 along with the error message and timestamp. This approach improves debugging and prevents exposing sensitive implementation details.

10. Screenshots

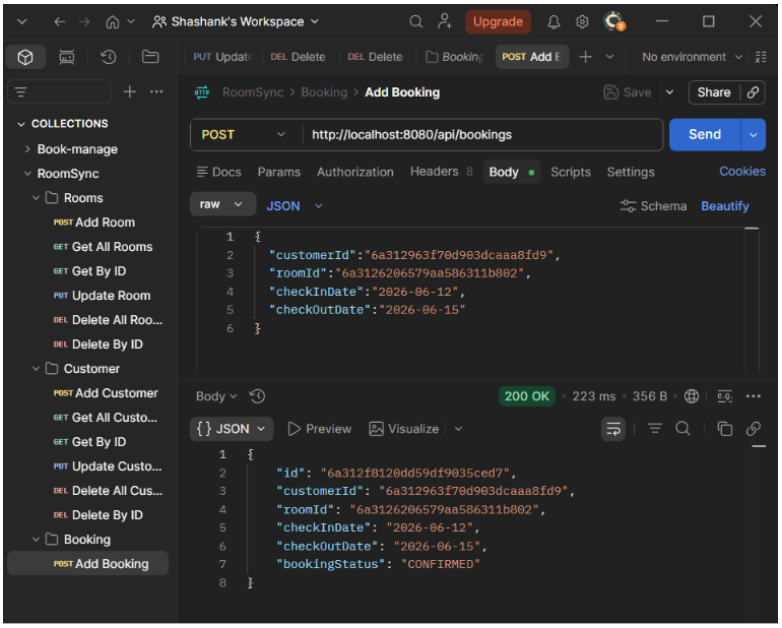
Add Room



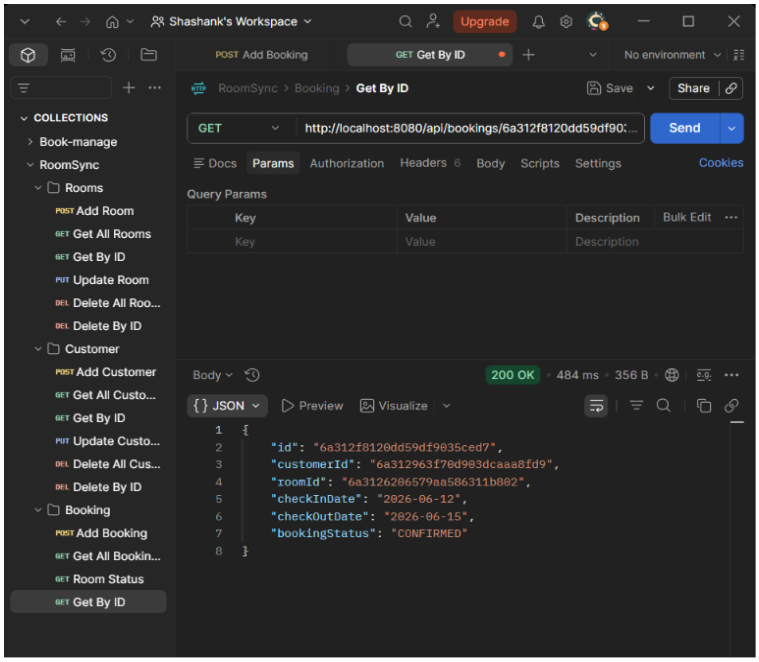
Add Customer



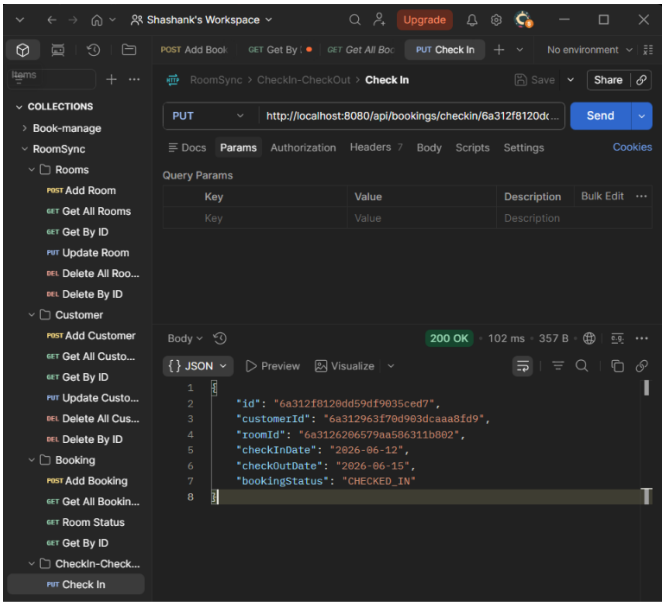
Create Booking



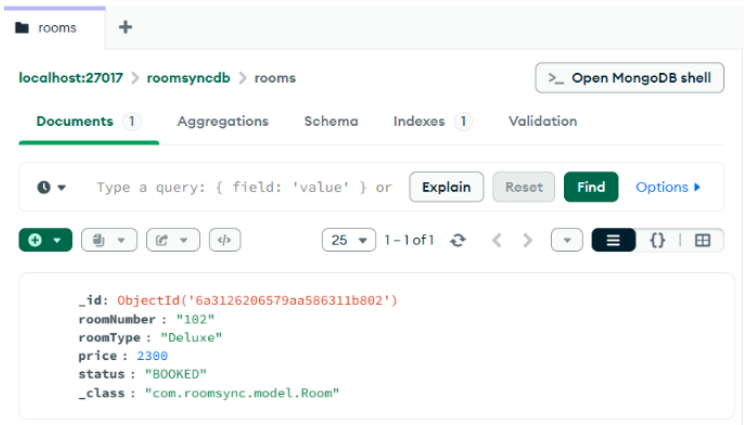
Get Booking By ID



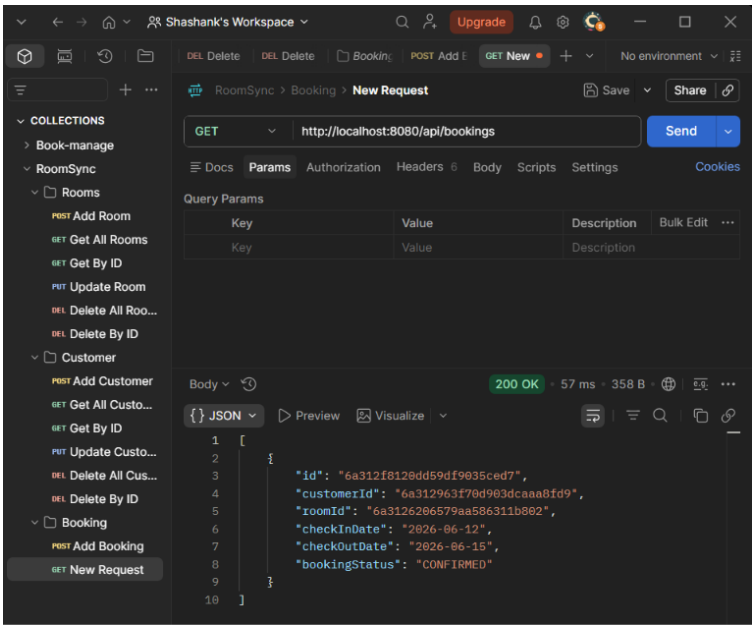
Check - In



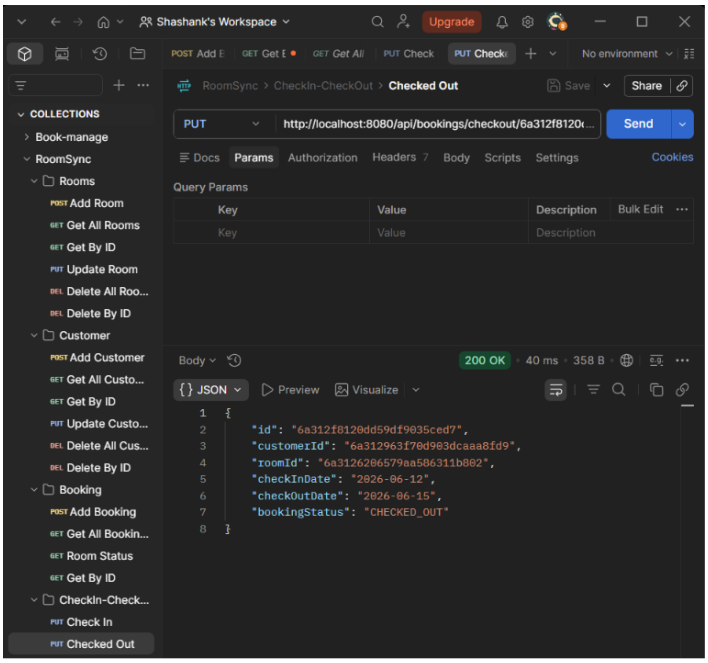
Booking Status()



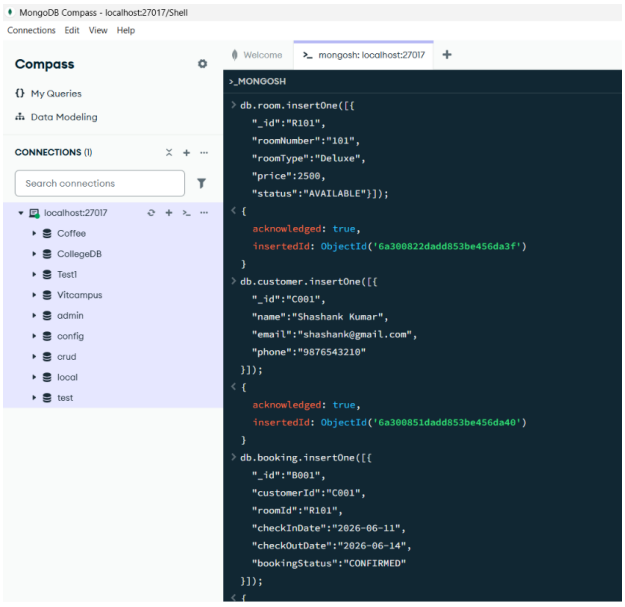
Get All Bookings



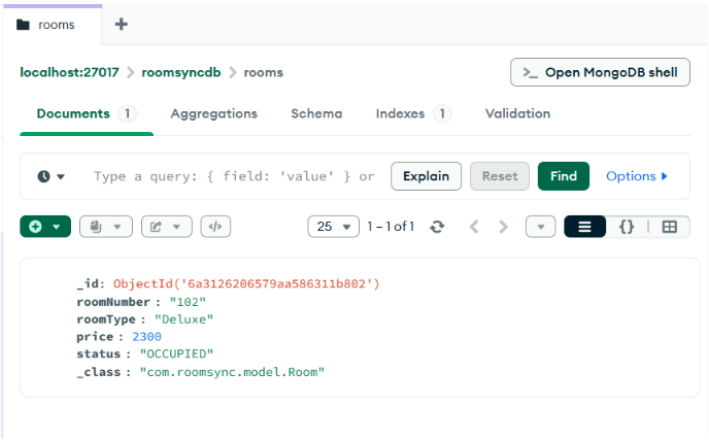
Check - Out



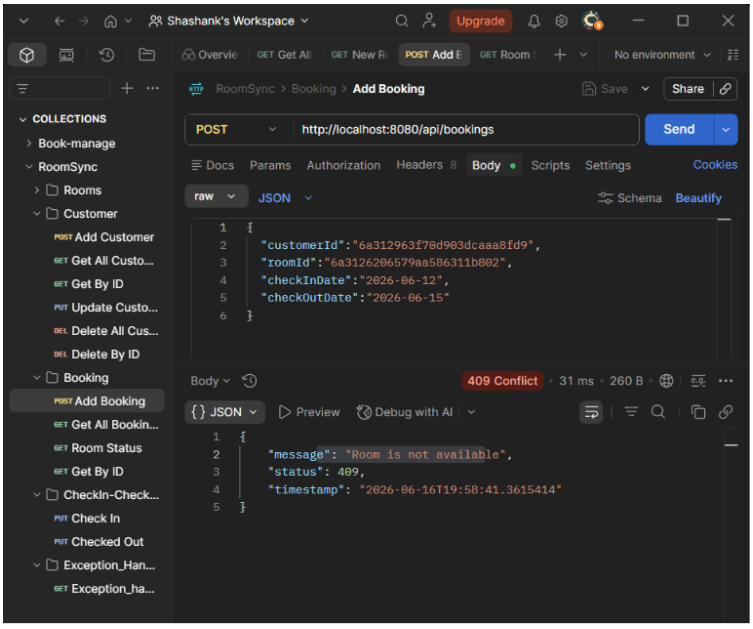
Database roomsyncdb



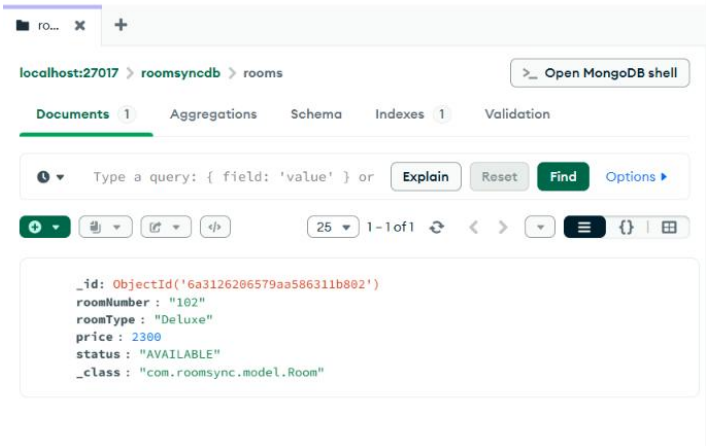
Check-In Status()



Dual Bookings Issue Resolved



Check Out Status()



//These are not all the screenshots of the project, rest are uploaded in the Final Project folder of the GitHub Repository

GitHub Repository Link : (Ctrl+Click)

https://github.com/Shashank14105/24BAI10032_ShashankKumar/tree/main/Final%20Project

10. Conclusion

The project successfully demonstrates CRUD operations, entity relationships, exception handling, business logic, and MongoDB integration using Spring Boot. The project also provided hands-on experience in designing REST APIs and solving real-world problems.

11. Future Enhancements

- User Authentication and Authorization
- Payment Integration
- Room Availability Tracking
- Reports and Analytics
- Admin Dashboard

12. Summary

RoomSync is a backend-based Hotel Reservation and Management System developed using Spring Boot and MongoDB to manage rooms, customers, and bookings efficiently. The project implements complete CRUD operations and follows a layered architecture consisting of Controller, Service, Repository, and Model components. Business logic such as room booking, check-in, check-out, booking cancellation, room status management, and date validation has been incorporated to simulate real-world hotel operations. Custom exceptions and global exception handling were implemented to provide meaningful error responses and improve reliability. Throughout the development process, several challenges related to booking status transitions, date validation, and room availability were identified and resolved. The project provided practical experience in REST API development, MongoDB integration, exception handling, and implementing business rules using Spring Boot.

Submitted to FacePrep as part of the Capstone Project for the Summer Internship on MongoDB